

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

May 16, 2026



Table of Contents

Ariane 5 Flight Failure (1996)

- On 4 June 1996, the maiden flight of the Ariane 5 launcher failed.
- About 40 seconds after launch, at an altitude of 3700 m:
 - The rocket veered off its flight path
 - Broke apart
 - Exploded
- Cause: A floating-point to integer conversion error.
- Result: Software exception led to system shutdown.

Lesson: Small numerical errors can lead to catastrophic system failures.

Specification Example

- Specification:
“In 1992, Mary received an invitation to attend a kindergarten in Winona, Minnesota, along with others born in '88.”
- Interpretation issue:
 - Does '88 mean 1988 or 1888?
- Truth:
 - Mary was 104 years old.

Lesson: Ambiguous specifications lead to incorrect system behaviour.

Key Idea

If you cannot precisely specify it, you cannot correctly implement or verify it.

- You must know what the system should do before checking correctness.
- Bugs often originate from unclear or incomplete specifications.

What is a Specification?

Definition

- A specification defines the product being created.

Types of Requirements

• Functional Requirements

- Describe system features and behavior
- Example (Word Processor):
 - Save, print, spell check, change font

• Non-Functional Requirements

- Describe system constraints
- Examples:
 - Security, reliability, usability, performance, platform

Why Verification and Validation?

- **Verification:** Are we building the product right?
- **Validation:** Are we building the right product?

Verification ensures correctness with respect to the specification.

Validation ensures the system meets user needs.

Common Sources of Errors

- Ambiguous specifications
- Incorrect assumptions
- Numerical errors (e.g., floating-point)
- Incomplete requirements
- Human misunderstanding

Incident (September 2004)

- Air traffic controllers in the Los Angeles area lost voice communication with approximately 800 aircraft.
- Around 10 aircraft came dangerously close to each other.
- Cause: Radio communication system shutdown.

Lesson: Design flaws in critical systems can lead to life-threatening situations.

Project Mercury (FORTRAN)

- Faulty code:
 - `DO I = 1.10` (Incorrect)
 - `DO I = 1,10` (Correct)
- Issue:
 - The loop executed only once instead of ten times.
- Discovered during:
 - Analysis of inaccurate computation results

Lesson: Small syntax errors can lead to significant logical faults.

- **F-18 Crash**

- Cause: Missing exception handling
- A condition assumed impossible occurred
- No **else** branch was implemented

- **F-16 Simulation Bug**

- The aircraft flipped over when crossing the equator
- Cause: Missing negative sign for south latitude

Lesson: Incorrect assumptions and incomplete logic can cause system failures.

What is a Software Bug?

A software bug occurs when:

- The software does not do something that the specification requires.
- The software does something that the specification forbids.
- The software behaves in a way not described in the specification.
- The software fails to meet implicit or expected requirements.
- The software is difficult to use, inefficient, or hard to understand.

Where Do Bugs Come From?

Distribution of Bugs Across SDLC Phases

- Specification: $\approx 55\%$
- Design: $\approx 25\%$
- Implementation (Coding): $\approx 15\%$
- Other: $\approx 5\%$

Key Insight

Most bugs originate **before coding even begins.**

Relative Cost of Bugs

Key Idea: Bugs found later cost significantly more to fix.

- Cost increases approximately **10x** at each stage.

Example:

- Specification phase: \$1
- Design phase: \$10
- Implementation phase: \$100
- After release: \$1000

Insight

Early detection of defects is critical to reducing cost and risk.

Sources of Software Problems

- **Requirements Definition**

- Erroneous, incomplete, or inconsistent requirements

- **Design**

- Fundamental flaws in system architecture

- **Implementation**

- Programming errors, hardware faults, malicious code

- **Support Systems**

- Faulty tools, compilers, debuggers, or languages

- **Testing**

- Incomplete testing, poor verification, debugging mistakes

- **Evolution (Maintenance)**

- Introducing new bugs while fixing old ones
- Increasing system complexity over time

Adverse Effects of Faulty Software (1)

- **Communication Systems**

- Data loss, corruption, or non-delivery

- **Space Applications**

- Mission failure, financial loss, loss of life

- **Defense and Warfare**

- Misidentification of friend vs. foe

- **Transportation**

- Accidents, delays, loss of control (e.g., braking failure)

- **Safety-Critical Systems**

- Injuries or fatalities

- **Electric Power Systems**

- Outages, injuries, long-term hazards (e.g., radiation exposure)

Adverse Effects of Faulty Software (2)

- **Financial Systems**

- Fraud, privacy violations, system shutdowns, incorrect calculations

- **Election Systems**

- Incorrect or manipulated results

- **Correctional Systems (Jails)**

- Unauthorized releases, system failures in locks

- **Law Enforcement**

- False arrests, wrongful imprisonment

Faulty software is not just a technical issue —
it has **real-world consequences**.

Verification and Validation are essential for building
safe, reliable, and trustworthy systems.

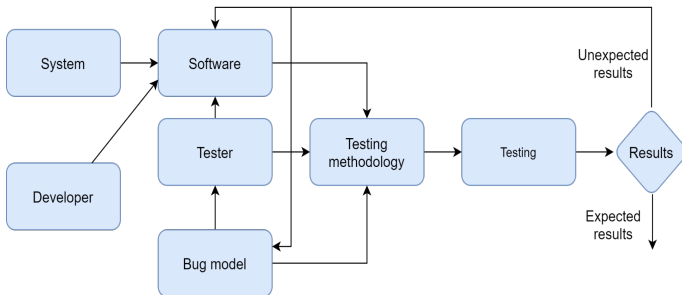


Figure 1: Software Testing Model

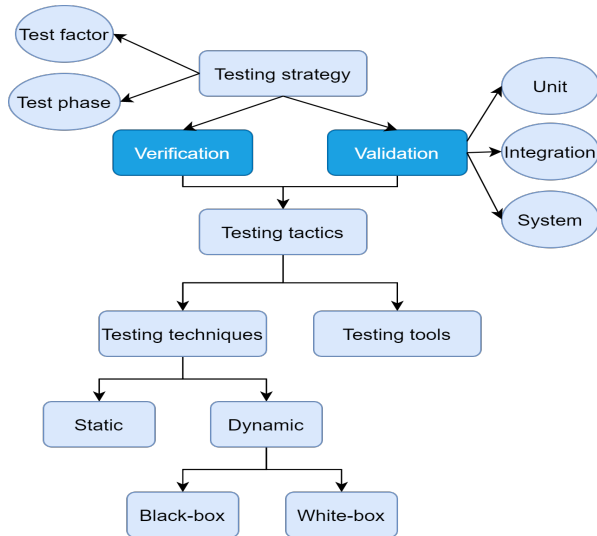


Figure 2: Testing Methodology

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`